Module 4 – Introduction to DBMS

Introduction to SQL

Name-- Komal Nafde

1. Whatis SQL, and why is it essential in database management?

Ans-SQL (Structured Query Language) is the standard language used to communicate with relational databases. It allows you to store, retrieve, update, and manage data efficiently.

SQL is used with database systems such as MySQL, PostgreSQL, Microsoft SQL Server, and Oracle Database.

What SQL Does

SQL helps users perform operations like:

Creating databases and tables

Inserting data

Retrieving information

Updating records

Deleting data

Managing permissions and security

Why SQL Is Essential in Database Management

1. Organizes Data Efficiently

SQL works with relational databases, where data is stored in structured tables. This organization makes data easy to manage and analyze.

2. Fast Data Retrieval

SQL can quickly search and filter large amounts of data using queries.

Example uses:

Banking transactions

E-commerce orders

School records

Hospital systems

3. Standardized Language

SQL is widely accepted across database platforms, making it easier for developers and organizations to work with different systems.

4. Supports Data Integrity

SQL databases enforce rules such as:

Primary keys

Foreign keys

Constraints

These help maintain accurate and consistent data.

5. Enables Multi-User Access

Database systems using SQL allow many users and applications to access and modify data safely at the same time.

6. Security and Access Control

SQL provides commands to control who can view or modify data.

Example:

GRANT SELECT ON employees TO analyst;

7. Critical for Modern Applications

Most modern software applications rely on SQL databases, including:

Websites

Mobile apps

Enterprise systems

Analytics platforms

2. Explain the difference between DBMS and RDBMS.

Ans-

Main Differences

Feature	DBMS	RDBMS
Full Form	Database Management system	Relational Database Management System
Data Storage	Stores data as files	Stores data in tables (rows & columns)
Relationship Between Data	Usually not supported	Supported using keys
Normalization	Limited	Supports normalization to reduce redundancy
Data Redundancy	Higher	Lower
Security	Basic	More advanced
Multi-user Support	Limited	Strong multi-user support
Constraints	Usually absent	Supports constraints (Primary Key, Foreign Key, etc.)
SQL Support	May or may not use SQL	Usesns- SQL extensively
Scalability	Suitable for small systems	Suitable for large applications

3. Describe the role of SQL in managing relational databases.

Ans-

SQL (Structured Query Language) is the standard language used to interact with and manage relational databases, which store data in structured tables made up of rows and columns.

Its role can be understood through the main tasks it enables:

1. Defining database structure (DDL – Data Definition Language)

SQL is used to create and modify the structure of databases. This includes:

Creating tables (CREATE TABLE)

Changing table structure (ALTER TABLE)

Removing tables (DROP TABLE)

This is how a relational database's schema is built and maintained.

2. Managing data (DML – Data Manipulation Language)

SQL allows users to insert, update, delete, and retrieve data:

Adding data (INSERT)

Changing data (UPDATE)

Removing data (DELETE)

Reading data (SELECT)

This is the core way applications and users interact with stored information.

3. Querying data efficiently

SQL's most powerful feature is querying. It can:

Filter data using conditions (WHERE)

Combine data from multiple tables (JOIN)

Group and summarize data (GROUP BY, HAVING)

Sort results (ORDER BY)

This makes it possible to extract meaningful insights from large datasets.

4. Controlling access and security (DCL – Data Control Language)

SQL manages permissions and security:

Granting access (GRANT)

Revoking access (REVOKE)

This ensures only authorized users can view or modify data.

5. Managing transactions (TCL – Transaction Control Language)

SQL ensures data consistency through transactions:

Saving changes (COMMIT)

Undoing changes (ROLLBACK)

Setting checkpoints (SAVEPOINT)

This is essential for reliability, especially when multiple operations must succeed or fail together.

4. What are the key features of SQL?

Ans-

SQL has several key features that make it the standard language for working with relational databases:

1. Simple and declarative language

SQL is non-procedural, meaning you specify what you want (e.g., “get all customers”) rather than how to retrieve it.

This makes it easier to learn and use compared to low-level programming approaches.

2. Data definition capability (DDL)

SQL lets you define and modify database structures such as:

Tables

Indexes

Schemas

This helps design and maintain the organization of data.

3. Data manipulation (DML)

It provides commands to work with stored data:

Insert new records

Update existing records

Delete records

Retrieve data using queries

4. Powerful querying and filtering

SQL supports advanced querying features such as:

Conditions (WHERE)

Sorting (ORDER BY)

Grouping (GROUP BY)

Joining multiple tables (JOIN)

This allows complex data retrieval from large datasets.

5. Data integrity and constraints

SQL enforces rules to ensure accuracy and consistency, such as:

Primary keys (unique identification)

Foreign keys (relationships between tables)

NOT NULL, UNIQUE, CHECK constraints

6. Transaction control (ACID compliance support)

SQL supports reliable transaction processing:

Atomicity (all or nothing)

Consistency (valid data rules)

Isolation (independent transactions)

Durability (permanent changes)

7. Security and access control

SQL allows administrators to control user permissions using commands like GRANT and REVOKE, ensuring data protection.

8. Portability and standardization

SQL is a standardized language (with ANSI/ISO standards), so it works across most relational database systems like MySQL, PostgreSQL, Oracle, and SQL Server with minimal changes.

4. What are the basic components of SQL syntax?

Ans-

The basic components of SQL (Structured Query Language) syntax are the standard clauses and keywords used to create, retrieve, update, and delete data in a database.

1. SQL Keywords

These are reserved words that define the operation to perform.

Examples:

SELECT – retrieves data

FROM – specifies the table

WHERE – filters rows

INSERT INTO – adds new records

UPDATE – modifies existing records

DELETE FROM – removes records

CREATE TABLE – creates a table

ALTER TABLE – modifies a table structure

DROP TABLE – deletes a table

2. Table Names

The name of the table on which the SQL statement operates.

Example:

SELECT * FROM Students;

Here, Students is the table name.

3. Column Names

The fields or attributes within a table.

Example:

SELECT Name, Age FROM Students;

Here, Name and Age are column names.

4. Expressions

Calculations or operations performed on column values.

Example:

SELECT Salary * 12 AS AnnualSalary

FROM Employees;

5. Conditions

Used with WHERE to filter records.

Example:

```
SELECT * FROM Students
WHERE Age > 18;
```

Common operators:

=, >, <, >=, <=, <>

AND, OR, NOT

LIKE, IN, BETWEEN

6. Clauses

Clauses are building blocks of SQL statements.

Common Clauses:

SELECT

FROM

WHERE

GROUP BY

HAVING

ORDER BY

Example:

```
SELECT Department, COUNT(*)
```

```
FROM Employees
WHERE Salary > 30000
GROUP BY Department
HAVING COUNT(*) > 5
ORDER BY Department;
```

7. Functions

Built-in functions for calculations and aggregations.

Examples:

COUNT()

SUM()

AVG()

MIN()

MAX()

8. Aliases

Temporary names assigned to columns or tables.

Example:

```
SELECT Name AS StudentName
FROM Students;
```

9. Literals

Fixed values used in queries.

Examples:

Numbers: 100

Strings: 'Komal'

Dates: '2026-05-13'

10. Comments

Used to document SQL code.

-- Single-line comment

/* Multi-line
  comment */

5. Write the general structure of an SQL SELECT statement.

Ans-

General Structure of an SQL SELECT Statement

SELECT column_name1, column_name2, ...

FROM table_name

WHERE condition

GROUP BY column_name

HAVING condition

ORDER BY column_name ASC|DESC;

Explanation of Each Clause

SELECT – Specifies the columns to be retrieved.

FROM – Specifies the table from which data is fetched.

WHERE – Filters rows based on a condition.

GROUP BY – Groups rows having the same values.

HAVING – Applies conditions to groups.

ORDER BY – Sorts the result in ascending (ASC) or descending (DESC) order.

Example

SELECT Name, Salary

FROM Employee

WHERE Salary > 30000

GROUP BY Name

HAVING COUNT(Name) > 1

ORDER BY Salary DESC;

6. Explain the role of clauses in SQL statements.

Ans-

Role of Clauses in SQL Statements

In SQL, a clause is a part of a statement that performs a specific function.

Clauses are combined to form complete SQL queries and determine what data is selected, filtered, grouped, and sorted.

1. SELECT Clause

Specifies the columns to retrieve from a table.

SELECT Name, Age

2. FROM Clause

Specifies the table that contains the data.

FROM Students

3. WHERE Clause

Filters rows based on a condition.

WHERE Age > 18

4. GROUP BY Clause

Groups rows with the same values in specified columns.

GROUP BY Department

5. HAVING Clause

Filters grouped data after GROUP BY.

HAVING COUNT(*) > 5

6. ORDER BY Clause

Sorts the result set in ascending or descending order.

ORDER BY Salary DESC

7. LIMIT Clause (used in some SQL systems)

Restricts the number of rows returned.

LIMIT 10

Clause Roles in the Example

SELECT → shows department and employee count.

FROM → reads data from Employees table.

WHERE → includes only employees with salary > 30000.

GROUP BY → groups records by department.

HAVING → keeps groups with more than 2 employees.

ORDER BY → sorts the output by department.

7. What are constraints in SQL? List and explain the different types of constraints.

Ans-

Constraints are rules applied to table columns in a database to ensure the accuracy, validity, and integrity of the data.

They restrict the type of data that can be inserted, updated, or deleted in a table.

Constraints help maintain:

Data integrity

Data consistency

Data reliability

Types of Constraints in SQL

1. NOT NULL

Ensures that a column cannot contain NULL (empty) values.

Example

```
CREATE TABLE Student (
  RollNo INT,
  Name VARCHAR(50) NOT NULL
);
```

Explanation

The Name column must have a value for every record.

2. UNIQUE

Ensures that all values in a column are different.

Example

```
CREATE TABLE Student (
  Email VARCHAR(100) UNIQUE
);
```

Explanation

No two students can have the same email address.

3. PRIMARY KEY

Uniquely identifies each record in a table.

It is a combination of NOT NULL and UNIQUE.

Example

```
CREATE TABLE Student (
   RollNo INT PRIMARY KEY,
   Name VARCHAR(50)
);
```

Explanation

Each student must have a unique and non-null RollNo.

4. FOREIGN KEY

Creates a relationship between two tables.

It ensures that values in one table match values in another table.

Example

```
CREATE TABLE Department (
   DeptID INT PRIMARY KEY
);

CREATE TABLE Student (
```

```
  RollNo INT PRIMARY KEY,
  DeptID INT,
  FOREIGN KEY (DeptID) REFERENCES Department(DeptID)
);
```

Explanation

DeptID in Student must exist in the Department table.

5. CHECK

Ensures that all values satisfy a specified condition.

Example

```
CREATE TABLE Student (
  Age INT CHECK (Age >= 18)
);
```

Explanation

Only values 18 or greater are allowed in the Age column.

6. DEFAULT

Assigns a default value if no value is provided.

Example

```
CREATE TABLE Student (
```

City VARCHAR(50) DEFAULT 'Bhopal'

);

Explanation

If City is not specified, 'Bhopal' is inserted automatically.

7. AUTO_INCREMENT / IDENTITY

Automatically generates sequential values (DBMS-specific).

Example (MySQL)

CREATE TABLE Student (

RollNo INT AUTO_INCREMENT PRIMARY KEY,

Name VARCHAR(50)

);

Explanation

RollNo is generated automatically for each new record.

8. How do PRIMARY KEY and FOREIGN KEY constraints differ?

Ans-

Difference Between PRIMARY KEY and FOREIGN KEY Constraints

Basis	PRIMARY KEY	FOREIGN KEY
Definition	A column or set of columns that uniquely identifies each row in a table.	A column or set of columns that creates a relationship between two tables.

Uniqueness	Values must be unique.	Values can be repeated.
NULL Values	Cannot contain NULL.	Can contain NULL (unless restricted).
Number Allowed	Only one PRIMARY KEY per table.	A table can have multiple FOREIGN KEYs.
Purpose	Ensures entity integrity by uniquely identifying records.	Ensures referential integrity by linking records between tables.
Reference	Defined within the same table.	References the PRIMARY KEY (or UNIQUE key) of another table.

9. Whatisthe role of NOT NULL and UNIQUE constraints?

Ans-

Role of NOT NULL and UNIQUE Constraints in SQL

Constraints are rules applied to table columns to maintain data accuracy and integrity. Two important constraints are NOT NULL and UNIQUE.

1. NOT NULL Constraint

The NOT NULL constraint ensures that a column cannot store NULL values. This means every record must contain a value for that column.

Role of NOT NULL

Prevents missing or empty values.

Ensures that important fields always contain data.

Improves data reliability.

Example

```
CREATE TABLE Student (
  StudentID INT,
  Name VARCHAR(50) NOT NULL,
  City VARCHAR(30)
);
```

In this example, the Name column must contain a value for every row.

2. UNIQUE Constraint

The UNIQUE constraint ensures that all values in a column are different from each other.

Role of UNIQUE

Prevents duplicate values.

Maintains uniqueness of data.

Useful for columns like email addresses, phone numbers, and usernames.

Example

```
CREATE TABLE Student (
  StudentID INT,
  Email VARCHAR(100) UNIQUE
);
```

9. Define the SQL Data Definition Language (DDL).

Ans-

Data Definition Language (DDL) is a subset of SQL used to define, modify, and remove the structure of database objects such as tables, views, indexes, and schemas.

DDL commands affect the database schema rather than the data stored in the tables.

Main DDL Commands

1. CREATE

Used to create new database objects.

```
CREATE TABLE Student (
  StudentID INT PRIMARY KEY,
  Name VARCHAR(50),
  Age INT
);
```

2. ALTER

Used to modify the structure of an existing table.

```
ALTER TABLE Student
ADD City VARCHAR(30);
```

3. DROP

Used to permanently delete a database object.

DROP TABLE Student;

4. TRUNCATE

Removes all rows from a table but keeps the table structure intact.

TRUNCATE TABLE Student;

5. RENAME

Changes the name of a database object.

RENAME TABLE Student TO Students;

Characteristics of DDL

Used to define the database schema.

Automatically commits changes in many database systems.

Changes table structures, not individual data values.

Can create and enforce constraints such as PRIMARY KEY, FOREIGN KEY, NOT NULL, and UNIQUE.

10. Explain the CREATE command and its syntax.

Ans-

The CREATE command is a Data Definition Language (DDL) command used to create new database objects such as tables, databases, views, indexes, and schemas.

Most commonly, CREATE TABLE is used to define a new table along with its columns, data types, and constraints.

Purpose of the CREATE Command

Creates a new table or other database object.

Defines column names and data types.

Applies constraints such as PRIMARY KEY, NOT NULL, and UNIQUE.

General Syntax of CREATE TABLE

```
CREATE TABLE table_name (
   column1 datatype [constraint],
   column2 datatype [constraint],
   column3 datatype [constraint],
   ...
);
```

Example

```
CREATE TABLE Student (
   StudentID INT PRIMARY KEY,
   Name VARCHAR(50) NOT NULL,
   Email VARCHAR(100) UNIQUE,
   Age INT
);
```

Explanation

StudentID INT PRIMARY KEY → Unique identifier for each student.

Name VARCHAR(50) NOT NULL → Name must be provided.

Email VARCHAR(100) UNIQUE → Duplicate emails are not allowed.

Age INT → Stores age values.

11. What is the purpose of specifying data types and constraints during table creation?

Ans-

When creating a table in SQL, each column is defined with a data type and optional constraints.

These definitions determine what kind of data can be stored and what rules that data must follow.

1. Purpose of Data Types

A data type specifies the kind of values a column can store.

Why Data Types Are Important

Ensure only valid data is entered.

Use storage space efficiently.

Improve query performance.

Help prevent data entry errors.

Examples

INT → whole numbers

VARCHAR(50) → text up to 50 characters

DATE → date values

DECIMAL(10,2) → numeric values with decimals

Example

Age INT,

Name VARCHAR(50),

DOB DATE

2. Purpose of Constraints

Constraints enforce rules on the data stored in a table.

Why Constraints Are Important

Maintain data accuracy and consistency.

Prevent invalid, duplicate, or missing values.

Define relationships between tables.

Common Constraints

PRIMARY KEY → uniquely identifies each row.

FOREIGN KEY → links one table to another.

NOT NULL → disallows null values.

UNIQUE → prevents duplicate values.

CHECK → enforces custom conditions.

DEFAULT → supplies a default value.

Example

```
CREATE TABLE Student (
  StudentID INT PRIMARY KEY,
  Name VARCHAR(50) NOT NULL,
```

```
    Email VARCHAR(100) UNIQUE,
    Age INT CHECK (Age >= 18)
);
```

This table definition ensures:

StudentID is unique and not null.

Name must be provided.

Email cannot be duplicated.

Age must be at least 18.

12. What is the use of the ALTER command in SQL?

Ans-

The ALTER command is a Data Definition Language (DDL) command used to modify the structure of an existing database object, mainly tables, without deleting and recreating them.

It allows changes in columns, constraints, and table properties.

Purpose of ALTER Command

The ALTER command is used to:

Add new columns to a table

Modify existing columns (data type or size)

Drop (remove) columns

Add or remove constraints

Rename columns or tables (in some DBMS)

Common Uses of ALTER Command

1. Add a Column

ALTER TABLE Student

ADD Address VARCHAR(100);

➞ Adds a new column Address to the Student table.

2. Modify a Column

ALTER TABLE Student

MODIFY Name VARCHAR(100);

➞ Changes the size of the Name column.

3. Drop a Column

ALTER TABLE Student

DROP COLUMN Age;

➞ Removes the Age column from the table.

4. Add a Constraint

ALTER TABLE Student

ADD CONSTRAINT unique_email UNIQUE (Email);

➞ Adds a UNIQUE constraint to the Email column.

13. How can you add, modify, and drop columns from a table using ALTER?

Ans-

1. Add a Column

The ADD clause is used to insert a new column into a table.

Syntax:

ALTER TABLE table_name

ADD column_name datatype;

Example:

ALTER TABLE Student

ADD Address VARCHAR(100);

✓ This adds a new column Address to the Student table.

2. Modify a Column

The MODIFY (or ALTER COLUMN in some DBMS) is used to change the data type or size of an existing column.

Syntax:

ALTER TABLE table_name

MODIFY column_name new_datatype;

Example:

ALTER TABLE Student

MODIFY Name VARCHAR(100);

✓ This increases the size of the Name column.

3. Drop a Column

The DROP COLUMN clause is used to remove a column permanently from a table.

Syntax:

ALTER TABLE table_name

DROP COLUMN column_name;

Example:

ALTER TABLE Student

DROP COLUMN Age;

✓ This deletes the Age column from the table.

14. What is the function of the DROP command in SQL?

Ans-

The DROP command is a Data Definition Language (DDL) command used to permanently remove database objects such as tables, databases, views, or indexes from the database.

Once an object is dropped, both its structure and data are completely deleted and cannot be recovered easily.

Purpose of DROP Command

Deletes database objects permanently.

Removes both table structure and stored data.

Frees up database space.

Used when a table or database is no longer needed.

Syntax of DROP Command

1. Drop a Table

DROP TABLE table_name;

Example:

DROP TABLE Student;

➞ This deletes the Student table completely.

2. Drop a Database

DROP DATABASE database_name;

Example:

DROP DATABASE CollegeDB;

➞ This removes the entire database.

15. What are the implications of dropping a table from a database?

Ans-

The DROP TABLE command in SQL permanently removes a table from the database. This has several important consequences because it deletes both the table structure and its data.

1. Permanent Loss of Data

All records stored in the table are deleted permanently.

Data cannot be recovered unless a backup exists.

2. Removal of Table Structure

The table definition (columns, data types, constraints) is also deleted.

The table no longer exists in the database.

3. Loss of Dependencies

If other tables depend on this table (via foreign keys), those relationships are affected.

It may cause errors in related queries or applications.

4. Impact on Applications

Programs or queries using the table will fail after it is dropped.

Reports or transactions depending on the table will stop working.

5. Cannot Be Recovered Easily

Unlike DELETE, DROP is not easily reversible.

Recovery is only possible through database backup or restore.

6. Frees Database Space

Storage space used by the table is released.

Helps in cleaning unused or obsolete tables.

Example

DROP TABLE Student;

16. Define the INSERT, UPDATE, and DELETE commandsin SQL.

Ans-

These three commands are part of Data Manipulation Language (DML) in SQL. They are used to manage and modify the data stored inside database tables.

1. INSERT Command

The INSERT command is used to add new records (rows) into a table.

Purpose:

Insert new data into a table.

Syntax:

INSERT INTO table_name (column1, column2, ...)

VALUES (value1, value2, ...);

Example:

INSERT INTO Student (StudentID, Name, Age)

VALUES (1, 'Komal', 20);

2. UPDATE Command

The UPDATE command is used to modify existing records in a table.

Purpose:

Change existing data in one or more rows.

Syntax:

UPDATE table_name

SET column1 = value1, column2 = value2

WHERE condition;

Example:

UPDATE Student

SET Age = 21

WHERE StudentID = 1;

3. DELETE Command

The DELETE command is used to remove records from a table.

Purpose:

Delete one or more rows from a table.

Syntax:

DELETE FROM table_name

WHERE condition;

Example:

DELETE FROM Student

WHERE StudentID = 1;

17. What is the importance of the WHERE clause in UPDATE and DELETE operations?

Ans-

The WHERE clause in SQL is very important in UPDATE and DELETE statements because it helps to specify conditions for selecting particular rows to modify or remove.

Without the WHERE clause, the operation will affect all rows in the table, which can lead to serious data loss or incorrect updates.

1. Importance in UPDATE Command

In an UPDATE statement, the WHERE clause ensures that only specific records are modified.

Importance:

Prevents updating all rows accidentally.

Targets only required records.

Maintains data accuracy.

Example:

UPDATE Student

SET Age = 22

WHERE StudentID = 1;

➞ Only the student with StudentID = 1 will be updated.

Without WHERE:

UPDATE Student

SET Age = 22;

➞ All students' ages will be updated to 22 (dangerous).

2. Importance in DELETE Command

In a DELETE statement, the WHERE clause ensures that only selected records are deleted.

Importance:

Prevents deletion of entire table data.

Allows controlled removal of records.

Protects important data from accidental loss.

Example:

DELETE FROM Student

WHERE StudentID = 1;

➞ Only one record is deleted.

Without WHERE:

DELETE FROM Student;

➞ All records in the table are deleted.

18. What is the SELECT statement, and how is it used to query data?

Ans-

The SELECT statement is the most commonly used SQL command. It is used to retrieve (query) data from one or more tables in a database.

It is part of Data Query Language (DQL).

Purpose of SELECT Statement

Fetch data from a table.

Retrieve specific columns or all columns.

Apply conditions to filter data.

Sort and organize results.

Combine data from multiple tables (using joins).

Basic Syntax

SELECT column1, column2, ...

FROM table_name;

Example 1: Select All Columns

SELECT * FROM Student;

➞ Retrieves all records and all columns from the Student table.

Example 2: Select Specific Columns

SELECT Name, Age FROM Student;

➞ Retrieves only the Name and Age columns.

Example 3: Using WHERE Clause

SELECT * FROM Student

WHERE Age > 18;

➞ Retrieves only students whose age is greater than 18.

Example 4: Using ORDER BY

```
SELECT * FROM Student
ORDER BY Name ASC;
```

➞ Sorts results in ascending order by name.

19. Explain the use of the ORDER BY and WHERE clauses in SQL queries.

Ans-

1. WHERE Clause

The WHERE clause is used to filter records based on a specified condition.

Purpose:

Select only those rows that satisfy a condition.

Reduce unnecessary data retrieval.

Improve query accuracy.

Syntax:

```
SELECT column1, column2
FROM table_name
WHERE condition;
```

Example:

```
SELECT * FROM Student
WHERE Age > 18;
```

➞ This query retrieves only students whose age is greater than 18.

2. ORDER BY Clause

The ORDER BY clause is used to sort the result set in ascending or descending order.

Purpose:

Arrange data in a meaningful order.

Improve readability of results.

Sort data based on one or more columns.

Syntax:

SELECT column1, column2

FROM table_name

ORDER BY column_name ASC|DESC;

Example:

SELECT * FROM Student

ORDER BY Name ASC;

➞ This sorts all students alphabetically by name in ascending order.

Using WHERE and ORDER BY Together

Both clauses can be used in a single query.

Example:

SELECT * FROM Student

WHERE Age > 18

ORDER BY Name ASC;

➞ First filters students older than 18, then sorts them by name.

20. What is the purpose of GRANT and REVOKE in SQL?

Ans-

GRANT and REVOKE are Data Control Language (DCL) commands in SQL. They are used to manage user permissions and access control in a database.

1. GRANT Command

The GRANT command is used to give specific privileges (permissions) to users.

Purpose:

Allows users to access and perform operations on database objects.

Controls who can read, insert, update, or delete data.

Improves database security by assigning controlled access.

Syntax:

GRANT privilege_name

ON table_name

TO user_name;

Example:

GRANT SELECT, INSERT

ON Student

TO Rahul;

➞ This allows user Rahul to read and insert data into the Student table.

2. REVOKE Command

The REVOKE command is used to remove previously granted permissions from a user.

Purpose:

Withdraw access rights from users.

Enhance security by limiting database access.

Prevent unauthorized operations.

Syntax:

REVOKE privilege_name

ON table_name

FROM user_name;

Example:

REVOKE INSERT

ON Student

FROM Rahul;

➞ This removes the INSERT permission from user Rahul.

21. How do you manage privileges using these commands?

Ans-

1. Granting Privileges (GRANT)

The GRANT command is used to assign access rights to users.

How privileges are managed using GRANT:

Assign specific operations like SELECT, INSERT, UPDATE, DELETE.

Allow controlled access to database objects.

Can grant single or multiple permissions at once.

Syntax:

GRANT privilege_list

ON table_name

TO user_name;

Example:

GRANT SELECT, INSERT, UPDATE

ON Student

TO Amit;

➞ Amit can now read, add, and modify data in the Student table.

2. Revoking Privileges (REVOKE)

The REVOKE command is used to remove previously granted permissions.

How privileges are managed using REVOKE:

Removes specific access rights from users.

Restricts operations like insertion or deletion.

Helps maintain database security.

Syntax:

REVOKE privilege_list

ON table_name

FROM user_name;

Example:

REVOKE UPDATE

ON Student

FROM Amit;

➞ Amit can no longer modify records in the Student table.

3. Combined Privilege Management

DBA (Database Administrator) typically manages security like this:

Give required permissions only (GRANT)

Remove unnecessary permissions (REVOKE)

Ensure users have minimum required access (principle of least privilege)

22. What is the purpose of the COMMIT and ROLLBACK commands in SQL?

Ans-

COMMIT and ROLLBACK are Transaction Control Language (TCL) commands in SQL. They are used to manage transactions and control changes made to the database.

A transaction is a group of SQL operations executed as a single unit.

1. COMMIT Command

The COMMIT command is used to save all changes made during a transaction permanently in the database.

Purpose:

Makes changes permanent.

Ends the current transaction successfully.

Ensures data consistency.

Syntax:

COMMIT;

Example:

UPDATE Student

SET Age = 22

WHERE StudentID = 1;

COMMIT;

➞ The update becomes permanent in the database.

2. ROLLBACK Command

The ROLLBACK command is used to undo changes made during a transaction if an error occurs or if the user wants to cancel the operation.

Purpose:

Restores database to the previous state.

Cancels uncommitted changes.

Helps maintain data accuracy.

Syntax:

ROLLBACK;

Example:

UPDATE Student

SET Age = 22

WHERE StudentID = 1;

ROLLBACK;

➞ The update is cancelled, and the original data is restored.

23. COMMIT and ROLLBACK are Transaction Control Language (TCL) commands in SQL.

Ans-

COMMIT and ROLLBACK are part of Transaction Control Language (TCL) in SQL, and they are used to manage transactions in a database.

🔹 COMMIT

It is used to save all changes permanently made during a transaction.

Once COMMIT is executed, changes cannot be undone.

Example:

```
UPDATE Students SET marks = 85 WHERE id = 1;
COMMIT;
```

👉 Here, the updated marks are permanently saved in the database.

🔹 ROLLBACK

It is used to undo changes made in the current transaction.

It restores the database to the last saved state (before COMMIT).

Example:

```
UPDATE Students SET marks = 85 WHERE id = 1;
ROLLBACK;
```

👉 Here, the update is canceled and original data is restored.

24. Explain the concept of JOIN in SQL. What isthe difference between INNER JOIN, LEFTJOIN, RIGHT JOIN, and FULL OUTER JOIN?

Ans-

A JOIN in SQL is used to combine rows from two or more tables based on a related column between them (usually a primary key–foreign key relationship).

It allows us to retrieve meaningful data that is spread across multiple tables.

Example idea:

Students table

Marks table

Both tables can be linked using student_id.

🔷 Types of JOINs

1. 🔹 INNER JOIN

Returns only matching rows from both tables.

If there is no match, the row is not included.

Example:

```
SELECT Students.name, Marks.score
FROM Students
INNER JOIN Marks
ON Students.id = Marks.student_id;
```

✓ Only students who have marks will be shown.

2. 🔹 LEFT JOIN (LEFT OUTER JOIN)

Returns all rows from the left table (first table).

Matching rows from the right table are included.

If no match, NULL is shown for right table columns.

Example:

```
SELECT Students.name, Marks.score
FROM Students
LEFT JOIN Marks
ON Students.id = Marks.student_id;
```

✓ All students will be shown, even if they have no marks.

3. 🔹 RIGHT JOIN (RIGHT OUTER JOIN)

Returns all rows from the right table.

Matching rows from the left table are included.

If no match, NULL is shown for left table columns.

Example:

```
SELECT Students.name, Marks.score
FROM Students
RIGHT JOIN Marks
ON Students.id = Marks.student_id;
```

✓ All marks records will be shown, even if student details are missing.

4. 🔹 FULL OUTER JOIN

Returns all rows from both tables.

Matching rows are combined.

Non-matching rows show NULLs where data is missing.

Example:

```
SELECT Students.name, Marks.score
FROM Students
FULL OUTER JOIN Marks
ON Students.id = Marks.student_id;
```

✓ Shows everything from both tables.

25. How are joins used to combine data from multiple tables?

Ans-

In SQL, JOINs are used to combine related data from two or more tables by using a common column (such as a primary key in one table and a foreign key in another).

This helps retrieve meaningful, complete information that is split across different tables.

🔷 Step-by-step concept

Data is stored in separate tables to avoid duplication (normalization).

Each table contains a common linking column.

JOIN compares these columns.

Matching rows are combined into a single result set.

🔷 Example

Table 1: Students

id	name
1	Ravi
2	Neha
3	Amit

Table 2: Marks

student_id	score
1	85
2	90

🔷 Using INNER JOIN

```
SELECT Students.name, Marks.score
FROM Students
INNER JOIN Marks
ON Students.id = Marks.student_id;
```

🔹 Result:

name	score
Ravi	85
Neha	90

✓ Only matching records are combined.

🔷 How combination happens

SQL checks Students.id = Marks.student_id

If match found → data is merged in one row

If no match → depends on JOIN type:

INNER JOIN → ignored

LEFT JOIN → left table row kept with NULLs

RIGHT JOIN → right table row kept with NULLs

FULL JOIN → all rows included

🔷 Real-life use case

Imagine:

Students table = student details

Courses table = enrolled courses

Fees table = payment records

JOIN helps to show:

👉 “Which student paid fees and which course they enrolled in”

26. What is the GROUP BY clause in SQL? How is it used with aggregate functions?

Ans-

The GROUP BY clause in SQL is used to arrange rows that have the same values in one or more columns into groups. It is commonly used with aggregate functions to perform calculations on each group rather than on the entire table.

Purpose of GROUP BY

GROUP BY helps in:

Grouping similar data together.

Applying aggregate functions to each group.

Generating summarized reports.

Common Aggregate Functions Used with GROUP BY

COUNT() – Counts the number of rows.

SUM() – Calculates the total value.

AVG() – Calculates the average value.

MAX() – Finds the highest value.

MIN() – Finds the lowest value.

Syntax

SELECT column_name, aggregate_function(column_name)

FROM table_name

GROUP BY column_name;

Example

Suppose we have a table named Employees:

Department	Salary
HR	30000
IT	50000
HR	35000
IT	60000

Sales 40000

Query

SELECT Department, SUM(Salary) AS Total_Salary

FROM Employees

GROUP BY Department;

Output

Department Total_Salary

HR 65000

IT 110000

Sales 40000

27. Explain the difference between GROUP BY and ORDER BY.

Ans-

Basis	GROUP BY	ORDER BY
Purpose	Groups rows having the same values into a single summary rows.	Sorts the result set in ascending or descending order.
Used With	Aggregate functions such as SUM(), COUNT(), AVG(), MAX(), MIN().	Any query result, whether aggregates are used or not.
Effect on Rows	Reduces multiple rows into grouped rows.	Does not reduce rows; only changes their order.
Default Order	No specific sorting order is guaranteed.	Sorts in ascending (ASC) order by default.

Optional Direction	Not applicable.	Can use ASC or DESC.
Position in Query	Comes before ORDER BY.	Usually the last clause in a SELECT statement.

28. What is a stored procedure in SQL, and how doesit differ from a standard SQL query?

Ans-

Definition

A stored procedure is:

Stored permanently in the database.

Compiled once and reused many times.

Executed by calling its name.

Capable of accepting input and returning output.

Syntax Example

```
CREATE PROCEDURE GetEmployeeDetails
AS
BEGIN
  SELECT * FROM Employees;
END;
```

Execution

```
EXEC GetEmployeeDetails;
```

Standard SQL Query

A standard SQL query is a single statement written and executed directly by the user.

```
SELECT * FROM Employees;
```

It runs immediately and is not saved in the database unless you store it manually.

Difference Between Stored Procedure and Standard SQL Query

Basis	Stored Procedure	Standard SQL Query
Storage	Saved in the database	Not stored permanently
Compilation time	Precompiled once	Parsed and compiled each time
Reusability	Can be executed repeatedly	Must be rewritten each time
Parameters	Supports input/output parameters	Usually no parameter interface
Logic	Can include loops, conditions, and variables	Limited to the SQL statement itself
Performance	Often faster for repeated use	May be slower when executed repeatedly
Security	Can grant execute permission without exposing tables	Users may need direct table access
Maintenance	Centralized business logic	Logic may be duplicated in many places

30. Explain the advantages of using stored procedures.

Ans-

Stored procedures provide several important benefits in database applications. They improve performance, security, and maintainability by storing reusable SQL logic directly inside the database.

1. Improved Performance

Stored procedures are compiled and optimized when created, so the database can reuse execution plans. This reduces parsing and compilation overhead and can make repeated operations faster.

2. Code Reusability

A stored procedure can be executed many times from different applications or modules. This avoids rewriting the same SQL statements repeatedly.

3. Reduced Network Traffic

Instead of sending multiple SQL statements from the application, the client sends a single procedure call. The database performs all required steps internally.

4. Enhanced Security

Users can be granted permission to execute a procedure without being given direct access to the underlying tables. This helps protect sensitive data.

5. Easier Maintenance

Business logic is centralized in one place. If rules change, you update the stored procedure rather than modifying SQL code in multiple applications.

6. Supports Complex Logic

Stored procedures can include variables, parameters, conditional statements (IF), loops, cursors, and exception handling.

7. Transaction Management

Procedures can group multiple statements into a single transaction and use COMMIT and ROLLBACK to maintain data integrity.

8. Consistency

Using a single stored procedure ensures all applications follow the same business rules and validation logic.

9. Parameterization

Stored procedures can accept input parameters and return output parameters, making them flexible and reusable for different inputs.

10. Better Productivity

Developers can encapsulate common database operations into procedures, reducing coding effort and improving reliability.

31. What is a view in SQL, and how is it different from a table?

Ans-

Definition

A view:

Is based on a SQL query.

Presents selected rows and columns from one or more tables.

Can simplify complex queries.

Can restrict access to sensitive data.

Syntax

CREATE VIEW Employee_View AS

SELECT EmployeeID, Name, Department

FROM Employees;

To use the view:

SELECT * FROM Employee_View;

Table in SQL

A table is a physical database object that stores data permanently in rows and columns.

Example:

```
CREATE TABLE Employees (
  EmployeeID INT PRIMARY KEY,
  Name VARCHAR(50),
  Department VARCHAR(30)
);
```

Difference Between View and Table

Basis	View	Table
Nature	Virtual table based on a query	Physical object that stores data
Data Storage	Usually does not store data separately	Stores actual data
Creation	Created using CREATE VIEW	Created using CREATE TABLE
Space Usage	Minimal storage for the query definition	Requires storage for all rows

Data Source object	Derived from one or more tables/views	Independent storage
Updates	Some views are updatable, some are not	Fully supports INSERT, UPDATE, DELETE
Security	Can expose only selected columns/rows	Contains complete data
Use Case	Simplifying queries and restricting access	Permanent data storage

32. Explain the advantages of using viewsin SQL databases.

Ans-

A view is a virtual table based on the result of a SELECT query. Views are widely used to simplify data access, improve security, and provide a consistent interface to underlying tables.

1. Simplifies Complex Queries

Views can encapsulate complex joins, filters, and calculations. Users can query the view as if it were a regular table.

```
CREATE VIEW EmployeeSummary AS
SELECT e.EmployeeID, e.Name, d.DepartmentName
FROM Employees e
JOIN Departments d ON e.DepartmentID = d.DepartmentID;
```

2. Enhances Security

Views can expose only selected columns and rows, hiding sensitive information such as salaries or personal details.

3. Provides Data Abstraction

Applications can depend on a view instead of the underlying table structure. If base tables change, the view can often be updated without changing application code.

4. Reusability

Common query logic is defined once and reused by many users or applications.

5. Consistency

All users querying the same view see data in a standardized format with the same calculations and filters.

6. Easier Maintenance

Changes to business logic can be made in the view definition rather than in many separate queries.

7. Custom Presentation

Views can rename columns, combine tables, and format data to make it easier to understand.

8. Restricts Data Access

Views can limit records to only those relevant to certain users or departments.

9. Supports Backward Compatibility

If the schema changes, views can preserve the old interface so existing applications continue to work.

10. Can Improve Performance (in Some Systems)

Some database systems support materialized views, which store precomputed results for faster access.

33. What is a trigger in SQL? Describe its types and when they are used.

Ans-

Definition

A trigger:

Is stored in the database.

Executes automatically when a specified event occurs.

Cannot be called directly like a stored procedure.

Is associated with a table or view.

Basic Syntax

```
CREATE TRIGGER trigger_name
AFTER INSERT
ON Employees
FOR EACH ROW
BEGIN
   -- Trigger logic
END;
```

Types of Triggers

1. BEFORE Trigger

Executes before the triggering event occurs.

Used for:

Data validation

Modifying values before they are stored

Preventing invalid operations

Example: Ensure salary is not negative before inserting a row.

2. AFTER Trigger

Executes after the triggering event has completed successfully.

Used for:

Audit logging

Updating summary tables

Sending notifications

Example: Insert a record into an audit table after an employee row is updated.

3. INSTEAD OF Trigger

Executes in place of the triggering event, commonly on views.

Used for:

Making complex views updatable

Custom handling of inserts, updates, or deletes on views

Triggers by Event Type

INSERT Trigger

Fires when new rows are inserted.

Use: Set default values, validate data, or log inserts.

UPDATE Trigger

Fires when existing rows are modified.

Use: Track changes, enforce rules, or update related tables.

DELETE Trigger

Fires when rows are deleted.

Use: Archive deleted rows or prevent unauthorized deletions.

34. Explain the difference between INSERT, UPDATE, and DELETE triggers.

Ans-

1. INSERT Trigger

An INSERT trigger is executed automatically whenever a new row is inserted into a table.

Purpose

Validate newly inserted data.

Generate audit records.

Update related tables.

Example Use

When a new employee is added to the Employees table, a trigger can record this action in an audit table.

2. UPDATE Trigger

An UPDATE trigger is executed whenever existing rows in a table are modified.

Purpose

Track changes to important fields.

Validate updated values.

Store old and new values for auditing.

Example Use

When an employee's salary is changed, the trigger can save both the old salary and the new salary in an audit table.

3. DELETE Trigger

A DELETE trigger is executed whenever rows are removed from a table.

Purpose

Archive deleted records.

Log deletion activity.

Prevent unauthorized deletions.

Example Use

When an employee record is deleted, the trigger can copy the deleted row to a backup table before removal.

35. What are control structures in PL/SQL? Explain the IF-THEN and LOOP control structures.

Ans-

In PL/SQL, control structures are statements that determine the order in which code is executed. They allow a program to make decisions, repeat actions, and handle different conditions. Control structures make PL/SQL programs more flexible and powerful.

Types of Control Structures in PL/SQL

Conditional Control Structures

IF-THEN

IF-THEN-ELSE

IF-THEN-ELSIF

Iterative Control Structures (Loops)

Simple LOOP

WHILE LOOP

FOR LOOP

Sequential Control

GOTO

NULL

IF-THEN Control Structure

The IF-THEN statement is used to execute a block of code only when a specified condition is true.

Syntax

```
IF condition THEN
  statements;
END IF;
```

Working

The condition is evaluated.

If it is TRUE, the statements inside the block are executed.

If it is FALSE or NULL, the statements are skipped.

Example

```
DECLARE
  num NUMBER := 10;
BEGIN
  IF num > 0 THEN
    DBMS_OUTPUT.PUT_LINE('Number is positive');
  END IF;
END;
```

/

Output

Number is positive

LOOP Control Structure

The LOOP statement repeatedly executes a set of statements until an EXIT statement is encountered.

Syntax

```
LOOP
  statements;
  EXIT WHEN condition;
END LOOP;
```

Working

Statements inside the loop are executed repeatedly.

After each iteration, the EXIT WHEN condition is checked.

If the condition is TRUE, the loop terminates.

Example

```
DECLARE
  i NUMBER := 1;
BEGIN
  LOOP
    DBMS_OUTPUT.PUT_LINE('Value of i = ' || i);
    i := i + 1;
    EXIT WHEN i > 5;
```

```
  END LOOP;
END;
/
```

Output

```
Value of i = 1
Value of i = 2
Value of i = 3
Value of i = 4
Value of i = 5
```

35. How do control structures in PL/SQL help in writing complex queries?

Ans-

Control structures in PL/SQL help in writing complex queries and programs by allowing the code to make decisions, repeat tasks, and execute statements conditionally. Without control structures, PL/SQL programs would be limited to executing statements in a simple top-to-bottom order.

How Control Structures Help

1. Decision Making

Using IF-THEN, IF-THEN-ELSE, and CASE, PL/SQL can choose different actions based on conditions.

Example:

```
IF salary > 50000 THEN
  bonus := salary * 0.10;
```

```
ELSE
  bonus := salary * 0.05;
END IF;
```

This allows business rules to be implemented directly in the program.

2. Repeating Operations

Loops (LOOP, WHILE LOOP, FOR LOOP) are used to execute a block of code multiple times.

Example:

```
FOR i IN 1..10 LOOP
  DBMS_OUTPUT.PUT_LINE(i);
END LOOP;
```

This is useful when processing multiple rows, generating reports, or performing calculations.

3. Processing Query Results

Control structures can be combined with cursors to process records one by one.

Example:

```
FOR rec IN (SELECT emp_name, salary FROM employees) LOOP
```

```
  IF rec.salary > 50000 THEN
    DBMS_OUTPUT.PUT_LINE(rec.emp_name || ' is highly paid');
  END IF;
END LOOP;
```

This makes it possible to apply logic to each row returned by a query.

4. Error Handling and Validation

Control structures help validate data before inserting or updating it.

Example:

```
IF age < 18 THEN
  DBMS_OUTPUT.PUT_LINE('Not eligible');
END IF;
```

5. Implementing Complex Business Logic

Applications often require multiple conditions and iterations. Control structures allow PL/SQL to handle these requirements efficiently.

Examples include:

Calculating salaries and bonuses

Updating inventory

Generating reports

Applying discounts

Validating transactions

36. What is a cursor in PL/SQL? Explain the difference between implicit and explicit cursors.

Ans-

A cursor in PL/SQL is a memory area used to store the result set returned by a SQL statement. It allows PL/SQL to process query results one row at a time.

Cursors are especially useful when a query returns multiple rows and each row needs to be processed individually.

Why Cursors Are Used

When a SELECT statement returns more than one row, PL/SQL needs a mechanism to:

Retrieve each row.

Store it temporarily.

Process rows one by one.

A cursor provides this mechanism.

Types of Cursors in PL/SQL

Implicit Cursor

Explicit Cursor

Implicit Cursor

An implicit cursor is automatically created and managed by PL/SQL for:

INSERT

UPDATE

DELETE

SELECT ... INTO statements that return a single row

The programmer does not need to declare or control it.

Example

```
DECLARE
  emp_name VARCHAR2(50);
BEGIN
  SELECT name
  INTO emp_name
  FROM employees
  WHERE emp_id = 101;

  DBMS_OUTPUT.PUT_LINE(emp_name);
END;
/
```

PL/SQL automatically opens, fetches, and closes the cursor.

Cursor Attributes

SQL%FOUND

SQL%NOTFOUND

SQL%ROWCOUNT

SQL%ISOPEN

Explicit Cursor

An explicit cursor is declared by the programmer for queries that return multiple rows.

The programmer manually:

Declares the cursor

Opens it

Fetches rows

Closes it

37. When would you use an explicit cursor over an implicit one?

Ans-

You would use an explicit cursor when a SQL query returns multiple rows and you need to process each row one at a time with custom logic.

An implicit cursor is sufficient when the SQL statement returns only one row (such as SELECT ... INTO) or when you are using INSERT, UPDATE, or DELETE.

When to Use an Explicit Cursor

Use an explicit cursor in the following situations:

1. When a Query Returns Multiple Rows

If you need to examine or process every row returned by a query.

```
CURSOR emp_cursor IS
  SELECT emp_name, salary FROM employees;
```

2. When Row-by-Row Processing Is Required

When each row needs conditional checks or calculations.

```
IF salary > 50000 THEN
  bonus := salary * 0.10;
END IF;
```

3. When You Need More Control

Explicit cursors let you manually:

OPEN the cursor

FETCH rows

CLOSE the cursor

This is useful when processing must pause, resume, or stop under specific conditions.

4. When Working with Cursor Attributes

You may need attributes such as:

cursor_name%FOUND

cursor_name%NOTFOUND

cursor_name%ROWCOUNT

cursor_name%ISOPEN

5. When Returning Data to Applications

Explicit cursors (especially REF CURSORs) are often used in stored procedures to return result sets to applications.

38. Explain the concept of SAVEPOINT in transaction management. How do ROLLBACK andCOMMIT interact with savepoints?

Ans-

A savepoint divides a transaction into smaller logical units.

SAVEPOINT savepoint_name;

After creating a savepoint, you can return to that point if needed.

Why Use SAVEPOINT?

SAVEPOINTs are useful when:

A transaction contains multiple insert, update, or delete operations.

You want to undo only the recent changes.

You want to preserve earlier successful operations.

Syntax

Create a Savepoint

```
SAVEPOINT sp1;
```

Roll Back to a Savepoint

```
ROLLBACK TO sp1;
```

Commit the Transaction

```
COMMIT;
```

Example

```
BEGIN
  INSERT INTO accounts VALUES (101, 5000);

  SAVEPOINT sp1;

  INSERT INTO accounts VALUES (102, 7000);

  SAVEPOINT sp2;

  DELETE FROM accounts WHERE account_id = 101;

  ROLLBACK TO sp2;  -- Undo only the DELETE

  COMMIT;         -- Permanently save remaining changes
END;
/
```

Result

Insert of account 101 remains.

Insert of account 102 remains.

Delete of account 101 is undone.

COMMIT permanently saves both inserts.

Interaction of SAVEPOINT with ROLLBACK

1. ROLLBACK

ROLLBACK;

Undoes the entire transaction.

Removes all savepoints.

3. ROLLBACK TO savepoint_name

ROLLBACK TO sp1;

Undoes changes made after sp1.

Keeps changes made before sp1.

Preserves sp1.

Interaction of SAVEPOINT with COMMIT

COMMIT;

Permanently saves all changes.

Erases all existing savepoints.

Starts a new transaction.

After COMMIT, you cannot roll back to earlier savepoints.

Example Flow

INSERT INTO table1 VALUES (1);

SAVEPOINT A;

INSERT INTO table1 VALUES (2);

SAVEPOINT B;

INSERT INTO table1 VALUES (3);

ROLLBACK TO B; -- Removes row 3 only

COMMIT; -- Saves rows 1 and 2 permanently

39. When is it useful to use savepointsin a database transaction?

Ans-

Situations Where SAVEPOINTs Are Useful

1. Large Transactions with Multiple Operations

When a transaction includes several INSERT, UPDATE, or DELETE statements.

Example:

Insert customer record

Insert order record

Insert payment record

If the payment step fails, you can roll back to the savepoint created before it, while keeping the customer and order records.

2. Error Recovery

If an error occurs during one step, you can undo only the problematic part and continue.

SAVEPOINT before_update;

UPDATE employees SET salary = salary * 1.10;

-- If error occurs

ROLLBACK TO before_update;

3. Complex Business Processes

Useful in workflows such as:

Banking transactions

Payroll processing

Inventory updates

Order processing

These often contain several independent steps.

4. Testing Data Changes

Developers can test a series of operations and roll back to a savepoint if results are not as expected.

5. Nested Operations in PL/SQL

In stored procedures, savepoints allow partial rollback inside loops or conditional logic.

Real-Life Example: Online Shopping

Suppose a transaction performs:

Insert customer details

Create order

Update stock

Process payment

A savepoint can be set after step 3. If payment fails, you can roll back only the payment step and handle the error appropriately.